01 Engineering objective and evidence

Accurate target position and heading, with fewer unnecessary movement and turning ticks.

This guide explains the preserved **7/10 submitted agent**. Each method has a specific job: recognize a reference, verify geometry, construct a route, follow it, recover when confidence falls, or align at the goal. The later A* reconstruction remains a separate study.

The decision after the first run

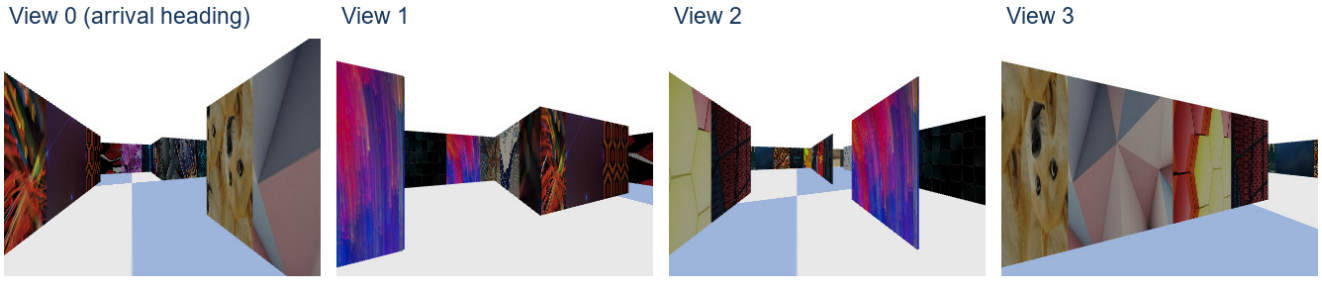
The first attempt reached the goal accurately and earned all six distance/heading points. Its low SPL made route connectivity and execution overhead the next priorities. The engineering decision was to preserve working arrival behavior while improving verified shortcuts and recovery. This is a rationale supported by the run and code, rather than an ablation proving which change produced each saved tick.

Recorded result	First attempt	Submitted source
Score	6/10	7/10
Executed ticks	2,451	1,418
SPL	0.358	0.619
Arrival distance	0.0157 m	0.0357 m
Heading error	0.359 degrees	0.094 degrees

The two recorded runs differ by **1,033 ticks (42.1%)**. The submitted run retained full pose points. This measured difference should not be attributed entirely to Dijkstra, PPO, or any one component.

Available information

The agent consumes public exploration RGB and recorded actions, SDK target images, camera calibration and current RGB observations. It does not read the hidden occupancy map, evaluator robot pose or true goal coordinates.



The four actual SDK target views describe the same goal pose. View 0 sets the required final heading.

Code evidence: SOURCE_PROVENANCE.json; teach_agent.py: VisualTeachIndex; preserved run summaries and diagnostics.

02 Image recognition

Find plausible references first, then ask whether the image evidence supports them.

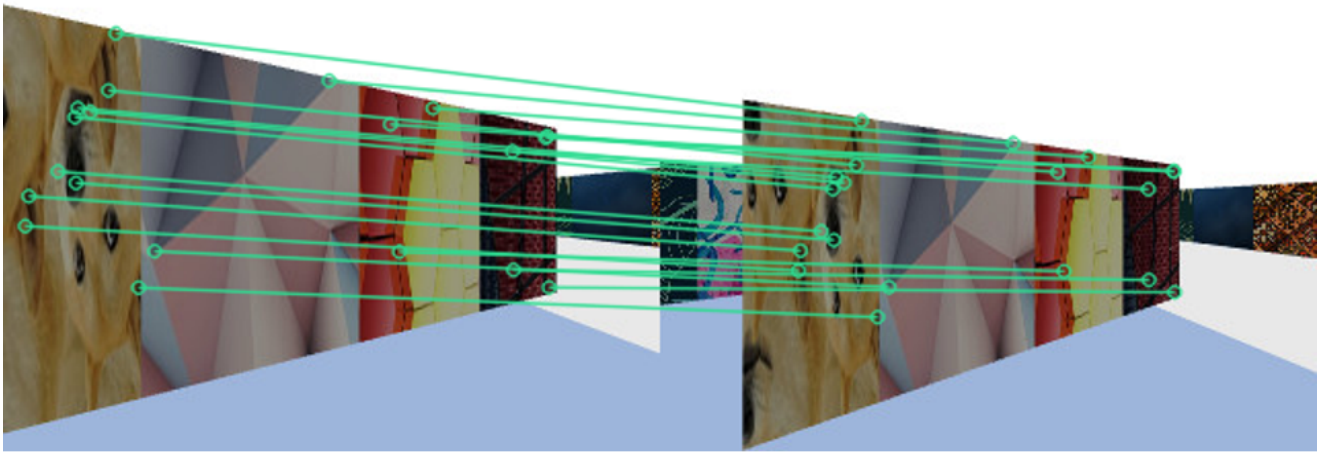
SIFT descriptors and nearest-neighbor ratio testing

SIFT extracts distinctive image locations and local descriptors. BFMatcher finds the two nearest descriptors in another image. A match passes when the nearest distance is below **0.7 times** the second-nearest distance. The intent is to reject ambiguous texture matches before estimating a transformation.

Homography RANSAC and target ranking

For every supplied target view, the index compares exploration features and fits a homography with RANSAC. Only a consistent set of inliers survives. The ranking uses inlier count divided by a penalty for median feature displacement, so a large match count alone does not establish alignment. The best-ranked target/reference pair supplies a goal reference and a view index. All four targets participate, while final arrival still uses view 0.

Supplied target view 3 Exploration reference 11646



SIFT matches + 2D RANSAC (illustration)

Actual target/reference correspondence illustration. The image shows a subset of homography-RANSAC inliers. A 2D image fit alone does not establish a traversable connection.

Thumbnails and TF-IDF visual words broaden revisit search

Small blurred thumbnails retrieve similar-looking frames. A separate bag-of-visual-words index clusters exploration SIFT descriptors into 64 visual words with MiniBatchKMeans. It downweights frequent words using TF-IDF, applies square-root normalization and retrieves cosine neighbors. Combining these proposals reduces dependence on a single image representation.

This is bag-of-visual-words retrieval, not VLAD or NetVLAD. It proposes candidates for later verification. It does not authorize a shortcut or a localization jump by itself.

Code evidence: teach_agent.py: _rank_target, match, setup; visual_word_candidates.py: candidates; word_visual_route.py: shorten_route.

03 Image-derived geometry and mapping

A similar texture needs geometric support before it becomes motion or a map connection.

Floor boundary and wall geometry

The reconstruction assumes an upright camera, a flat floor and vertical walls. It finds the visible wall/floor boundary and combines it with camera intrinsics to lift wall features into 3D. Depth and translation use **camera-height units**. The implementation uses the renderer's floor colours, so this is a scene assumption rather than a universally learned depth model.

PnP-RANSAC and refinement

Perspective-n-Point estimates the camera transformation from reconstructed reference points to live image pixels. RANSAC excludes inconsistent correspondences, and iterative PnP refines the retained inliers. The source rejects implausible vertical translation or camera tilt. Matching and pose estimation are distinct tasks: RANSAC improves robustness but does not plan a route.

Planar visual odometry and robust reprojection fitting

For consecutive images, planar odometry reconstructs wall points in both views, uses RANSAC to reject inconsistent planar matches, then estimates rotation with SVD and translation with a median residual. A separate upright planar-PnP verifier refines horizontal translation and yaw with a soft-L1 reprojection objective. It checks inlier error and conditioning before accepting a measurement.

Sparse pose graph and wall-axis heading prior

The map combines consecutive image-motion constraints and verified loop connections in a weighted sparse least-squares fit. A wall-axis estimate supplies heading information modulo 90 degrees. Visual constraints receive more trust than action-based fallbacks. The resulting coordinates support proposals and visualization, while the controller still needs fresh local measurements.

Independent verification and conditional learned matching

Shortcut checks use independent forward/reverse estimates, cycle closure, neighboring-frame support and image reprojection over multiple regions. SuperPoint/LightGlue supplies additional correspondences for configured weak-motion, return-verification or target-geometry paths. Geometry checks remain mandatory. Its presence in the source does not prove that every learned fallback accepted a constraint in the recorded run.

Code evidence: visual_pose.py: wall_points, relative_pose, image_pose_consistent; planar_odometry.py: estimate; planar_pnp.py: fit, from_points; spatial_map.py: build_map; wall_orientation.py: wall_axis; learned_matching.py: Matcher.

04 Route construction and Dijkstra

Improve the connections in the graph, then minimize the cost of a route through those supported connections.

A graph of recorded references and verified revisits

Each exploration frame becomes a node. Consecutive frames provide recorded edges. Appearance retrieval proposes nonconsecutive same-place links, which must pass feature, relative-pose, neighboring-view and whole-image checks. Public recordings start at the common spawn, so the search considers their start references together.

Dijkstra searches the available graph

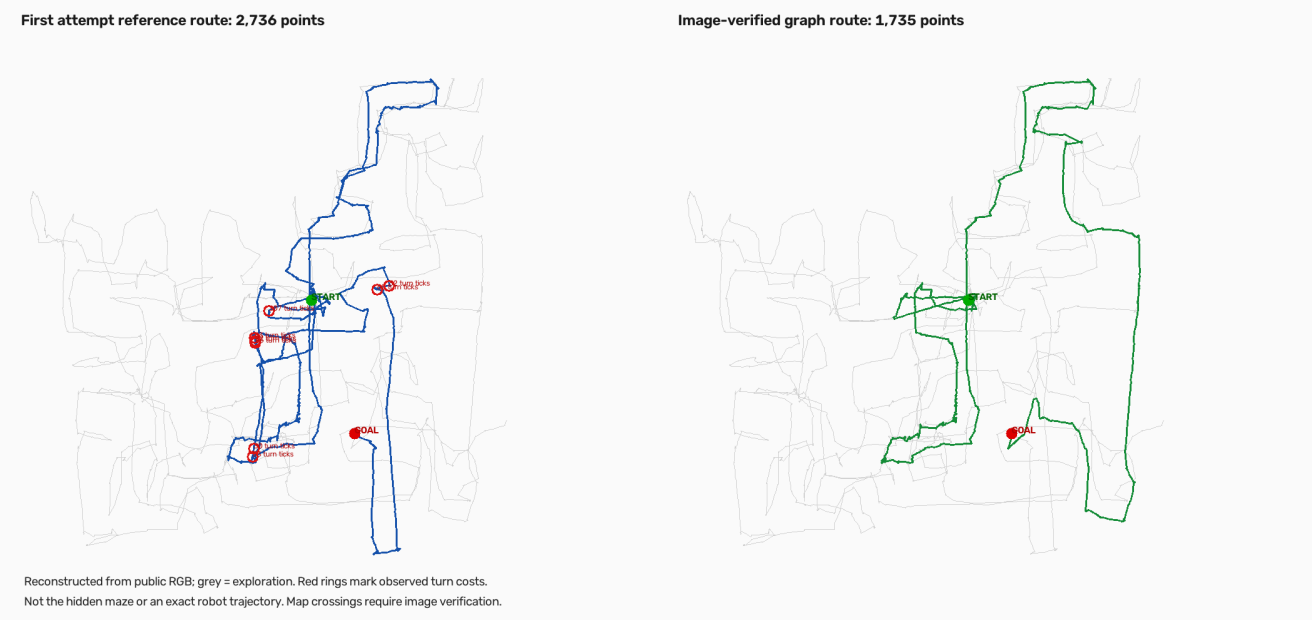
The implementation uses a priority queue and relaxes edges until it reaches the chosen goal reference. Initially consecutive edges cost 1 and verified same-place edges cost 0.2. The result minimizes those graph weights. If a useful corridor connection is absent or rejected, Dijkstra cannot invent it. Replacing Dijkstra with A* on the same graph and identical costs would not itself create a shorter optimum.

Return pruning and movement/turn estimates

Return verification examines out-and-back chains. It measures their image transforms in both directions, checks endpoint closure and visible overlap, then replans over supported alternatives. A return or added-motion edge uses the cost below, where p is an image-derived horizontal displacement and theta is yaw:

cost(p, theta) = max(0.2, (||p|| / 0.15 + |theta| / 0.06) / 5)

This combines estimated translation and rotation in units comparable with capture-edge costs. It is a planning proxy. It does not measure the ticks that noisy closed-loop execution will use. The optional image-link layer rejects a route replacement unless the modeled saving exceeds 5 percent, avoiding marginal switches to harder-to-localize views.



RGB-derived reference plans: 2,736 references in the first plan and 1,735 in the image-verified plan. These are reference counts, not executed ticks. Geometric line crossings do not establish passages.

Code evidence: word_visual_route.py: shorten_route; visual_return_route.py: search, proposals; planar_return_route.py: Verifier, cost, shorten; verified_map_policy.py: setup; route_motion_connections.py: attach_connections.

05 Closed-loop waypoint control

Use the map to choose a nearby direction, then use current images to check the motion.

Fresh localization and local path geometry

The live controller ranks nearby route references with thumbnails, then estimates their relative poses from SIFT features. Inlier count and pose bounds select a supported anchor. It builds a short local path from trusted motion edges and chooses a waypoint ahead. This limits dependence on a globally exact reconstruction.

Visual odometry and re-anchoring

Consecutive live RGB frames update the local pose with planar visual odometry. A command model supplies a temporary prediction when image motion is unavailable. A new supported reference corrects drift. Re-anchoring re-expresses a fresh pose through strong local edges instead of rewinding to an old reference. An inferred pose retains its limited prediction status.

A small PPO waypoint policy

The saved PPO network receives a **2D image-derived waypoint offset**. Its two hidden layers have 64 units each and an actor chooses among eight actions: forward, backward, left, right, or translation combined with steering. The deployed policy chooses the highest-logit action. It does not construct the global route or retrain on the new maze during navigation.

Geometric guards retain control authority

PPO proposals are eligible only during route following with fresh localization, moderate heading/bearing error and sufficient distance from final alignment. A one-step motion model rejects a move away from the waypoint or a turn that worsens bearing beyond tolerance. Otherwise geometric steering remains in control. This is guarded local action selection, not a complete MPC optimizer or an end-to-end learned navigation system.

Why keep the hybrid design?

The goal was to reduce small stop-turn-go corrections without handing uncertain localization or final check-in decisions to the learned policy. The source permits shallow simultaneous translation and steering when the pose and onward path support it. No isolated experiment here establishes a specific tick saving from PPO alone.

Code evidence: hybrid_rl_local_base.py: localize, act; hybrid_rl_reanchor.py: reexpress, localize; hybrid_rl_local.py: parent_mode; hybrid_rl_controller.py: Network, Controller.choose, guarded_choice.

06 Recovery and heading decisions

Treat repeated corrections as evidence of a localization or handoff problem, rather than continuing the same action indefinitely.

Oscillation monitoring

The monitor records forward/backward action signs and route progress. Four reversals in a 24-tick history, after at least 18 ticks at one route index, can trigger a visual recovery handoff. A cooldown limits repeated triggers. The route controller also has a progress timeout. A stationary route index is a diagnostic signal and does not prove the robot has not moved physically.

Visual recovery and bounded prediction

The fallback measures the current reference, tests nearby route references and can use resized feature variants when viewpoint changes make matching difficult. It performs small approach/heading actions when vision fails. An inferred pose transferred across a handoff has a limited lifetime. Fresh measured evidence remains necessary for the arrival checks.

Online translation calibration

Recovery prediction initially assumes a translation rate. The calibration layer measures consecutive public camera frames in both directions after pure forward/backward commands. It accepts high-inlier, low-side-slip, nearly straight measurements with good closure and takes a rolling median. After enough samples, it corrects the recovery prediction rate. This estimates camera motion from images; it does not read evaluator motion noise or true pose.

Intermediate headings are not final headings

At a strongly measured intermediate waypoint, the position-handoff layer can return to local path following when a trusted next edge exists. This avoids forcing the robot to recreate every captured camera heading. The final target heading remains a separate requirement. A forward-handoff guard also avoids replaying a completed edge when an old anchor ends behind current progress.

Remaining failure mechanism

Visual ambiguity can still lead to turns, failed measurements or backward corrections. The guards reduce the chance of indefinite repetition but do not eliminate it. A legitimate backward maneuver can also be part of a shorter repositioning action. Diagnose it with image evidence and the route/control state, not the action name alone.

Code evidence: oscillation_recovery.py: OscillationMonitor; hybrid_recovery_policy.py: act; pose_agent.py: recover_waypoint, affine_pose, act; odometry_recovery_policy.py: act; position_handoff_policy.py: act; word_route_policy.py: parent_mode.

07 Final alignment and diagnostics

Separate arrival at a reference from arrival at the supplied target pose.

The recorded goal frame can be offset from the target

The highest-ranked exploration frame identifies a useful goal neighborhood, but its camera position can differ from the supplied target pose. A conditional learned correspondence path estimates that target-relative offset and checks it in reverse. When supported, the controller aims at the inferred target position instead of stopping at the recorded reference.

Direct acquisition of target view 0

Near the end of the route, repeated direct SIFT/PnP matches to target view 0 can hand control to the final visual servo. Position and yaw are treated separately. The inherited controller uses bounded turning and translation actions and checks repeated measured final agreement with sufficient inliers before returning CHECKIN. Its internal tolerances are image-derived estimates, not access to the server's true distance or heading.

Why preserve this part?

The first run had already earned full pose points. The submitted run also did so, with 0.0357 m distance and 0.094-degree heading error. That evidence justified preserving arrival behavior while investigating the larger route/recovery cost. It does not guarantee the same precision on every run.

Diagnostics explain the decisions

The wrapper records the supplied targets, RGB observations, source hashes, inferred localization, chosen action and repeat count. It also logs handoffs and repeated steering reversals. It returns the underlying controller decision unchanged. These records support post-run diagnosis, while hidden evaluator poses remain outside the policy and its caches.

Submission and reproducibility

The code upload contains the preserved 7/10 policy/support modules with a portable SDK launcher and public-image preprocessing tools. It excludes credentials, exploration data, weights and derived caches. The external PPO checkpoint and official matcher weights are documented resources. Rebuilding image connections with different search budgets can change the route, so a source-only package is not a promise to reproduce exactly 1,418 ticks.

Code evidence: learned_goal.py: geometry; hybrid_rl_policy.py: setup, act; pose_agent.py: act; run_diagnostics.py: instrument; README.md and MODEL_SETUP.md; SOURCE_PROVENANCE.json.

08 Method roles and honest conclusions

The contribution is the engineering integration and diagnosis, with each claim tied to the evidence available.

Question	Method in the submitted agent
Which images may match?	SIFT ratio matches, thumbnails, TF-IDF visual words
Which correspondences agree?	Homography RANSAC and PnP-RANSAC
What motion do images support?	Wall/floor geometry, planar registration, robust reprojection fit
How do observations form a map?	Weighted sparse pose graph and verified loops
Which supported route is cheaper?	Dijkstra, verified returns, movement/turn cost proxy
How is that route executed?	Visual odometry, re-anchoring, geometric steering, guarded PPO
What happens when progress fails?	Oscillation detection, visual recovery, motion calibration and handoffs
When should the agent check in?	Target-relative alignment, target-view-0 servo and repeated measured agreement

What the results support

The preserved implementation reached the goal and earned 7/10 in its recorded run, with substantially fewer executed ticks than the first attempt. The map figure shows a shorter reference plan. These facts support an engineering improvement, but they do not establish a globally optimal physical route, universal maze performance or a per-method causal contribution.

Later reconstruction stays separate

The final slide proposes corridor contraction, landmark/ALT A* with movement/turn/reliability costs and sequence-based pose ranking while preserving the hybrid controller. It is a later prototype, not recovered source for an unavailable earlier run. The submitted 7/10 agent does not use A*. Established ingredients and an integration hypothesis alone do not establish first-ever research novelty.

References used in the presentation

Lowe (2004), SIFT, IJCV. Fischler and Bolles (1981), RANSAC, CACM. Schulman et al. (2017), PPO. Lindenberger et al. (2023), LightGlue, ICCV. Goldberg and Harrelson (2005), landmark A* / ALT, SODA, for the later reconstruction. Complete source links appear in the presentation speaker notes. The method descriptions above come from the preserved source inspection, not from a new navigation test.